Agora: one sentence, one world

Feature and Scope Technical Report

Agora 2.0 Runtime and Creator Pipeline

July 2, 2026

Abstract

Agora 2.0 converts a compact natural-language world brief into a package-backed, multiplayer pixel simulation. The current implementation is a durable workflow rather than a single blocking request: it creates a draft, queues generation, records revision artifacts, runs an art and QA worker, publishes a package, exposes that package through a strict catalog, and serves live sessions through REST and WebSocket APIs. The product promise is simple: one sentence can become one coherent world, but the engineering contract behind that promise is explicit, staged, validated, and database-backed.

1 Product Thesis

The phrase "Agora: one sentence, one world" describes the desired user experience: a creator provides a short scenario seed, and Agora produces a playable social simulation world with places, characters, items, rules, visual assets, and live human interaction.

The technical thesis is stricter:

- Natural language is only the input surface.
- JSON remains the authoring contract.
- SQLite `world_package.db` remains the runtime bundle.
- Pixel UI and live mode consume package-backed state, not a hidden frontend model.
- Generated worlds become public only after package, asset, browser, and visual QA gates pass.

2 System Scope

Agora 2.0 owns the runnable integration layer:

Area	Scope
Creator workflow	Browser and API workflow for draft creation, revision, art generation, QA, and publish.
World compiler	LLM-generated builder spec, registry resolution, compiled `world_config.json`, scenario materialization, and package creation.
Asset pipeline	Generated map, agent atlases, manifest feeds, live-ready bootstrap data, and strict `PIXEL READ`.
Package runtime	`world_package.db`, `package_meta.json`, materialized static workspace, access-code export, and pull by code.

Pixel catalog	Public latest-by-seed world records with validation gating.
Live runtime	DB-backed live sessions, REST state/actions, WebSocket movement input, authoritative deltas, and write-behind spatial persistence.
Operational validation	Backend startup smoke test, headless Firefox Pixel launch validation, Gemini MAP QA, regression scripts, and load tests.

3 End-to-End Flow

3.1 Draft Creation

The creator UI posts to `/api/world-builder/drafts` with a world name, genre, player count target, agent count target, focus, seed, and brief. The backend creates a durable draft directory under `output/world_creator_drafts/`, writes a manifest, creates revision `r001`, writes a placeholder status, queues a systemd user worker, and returns immediately.

This is a core design constraint. Long generation must not block the HTTP request path.

3.2 Generation Worker

The queued command is:

```
python -m agora_ui.world_builder generate-worker \
  --package-root ... --draft-id ... --revision-id ...
```

The generation worker uses `VertexJsonClient` and the world-builder node pipeline to create planner, rooms, items, roles, main characters, hooks, visual direction, and gameplay loops. The normalized builder spec is compiled through `build_world_pipeline()`, which resolves registry-backed kits and policies into concrete artifacts.

The worker emits the revision artifact set:

- `builder_spec.json`
- `planner.json`
- `rooms_spec.json`
- `items_spec.json`
- `agents_spec.json`
- `pixel_frontend_spec.json`
- `compiler_report.json`
- `compiler_critique.json`
- `world_config.json`
- `scenario/`
- `world_summary.md`
- `world_package.db`
- `status.json`

3.3 Art and QA Worker

The current art worker executes three major commands:

1. `python -m macro_ui.build_macro_ui`
2. `asset_pipeline/generate_world_asset_set.py`
3. `asset_pipeline/build_live_ready_feed.py`

It then repacks the revision package with only the assets matching the current `world_id` and `world_revision`. The generated map is injected into:

- `world_config` under the Pixel frontend map asset field
- `scenario/map_grid.json` under `map_visual.background_url`

This prevents the package from passing with assets that exist on disk but are not connected to the runtime render path.

3.4 Validation Gates

A revision becomes publish-ready only after:

- `PIXEL READ` passes for the isolated package workspace.
- Backend startup validation passes.
- Pixel UI launches in headless Firefox.
- The temporary validation package clears the `validation_probe` lifecycle.
- Gemini MAP QA accepts both the stitched map asset and browser screenshot.

If `PIXEL READ` fails once, the live-ready feed may be retried once. Other critical failures must hardfail rather than silently producing placeholders.

3.5 Publish

Publishing exports the revision-local `world_package.db`; it must not rebuild from raw config. The public package lands under:

```
output/package_exports/{access_code}/
  world_package.db
  package_meta.json
  materialized/
```

The exported package receives a 16-character access code and is visible to Pixel only if it passes public catalog rules.

4 Feature Inventory

Feature	Current behavior
One-brief generation	Converts a natural-language brief into a normalized builder spec and compiled runtime config.
Draft/revision history	Stores every revision with status, summary, package, and compiler artifacts.
Feedback revision	Creates a new revision from user feedback and prior summary context.
Registry-backed world assembly	Uses profiles, component kits, frontend affordances, item collections, inventory layers, property policies, and knowledge policies.
Protagonist-first worlds	Supports main-character-led worlds without requiring anonymous regular role groups.
Dynamic item catalog	Allows generated item catalogs while self-healing mandatory frontend affordance items.
Room-scoped visual metadata	Keeps room purpose, biome, decor, floor tile, wall tile, palette, and dimensions available to map and asset stages.
Package DB runtime	Stores files, metadata, and structured definitions inside `world_package.db`.
Public catalog gating	Filters by `PIXEL READ`, startup status, source label, probe flag, and latest-by-seed deduplication.
Live multiplayer	Uses REST for boot/actions/state and WebSocket for realtime movement with authoritative deltas.

5 Boundary Conditions

Agora 2.0 is intentionally not:

- A frontend-only sandbox.
- A JSON-only live runtime.
- A package-optional demo.
- A system that accepts placeholder art as success.
- A runtime where the browser owns inventory, trade, or authoritative position state.

The central boundary is that generated content may be creative, but runtime truth must stay explicit, materialized, validated, and package-backed.

6 Operational Requirements

- The main runtime uses the `new_py310` conda environment or the Python chosen by `resolve_runtime_python()`.
- Creator workers load `$HOME/.config/agora_ui_runtime.env`.
- Fresh creator regression expects `AGORA_AISTUDIO_API_KEY` and `AGORA_VERTEX_API_KEY`.

- FLUX must be launched through `scripts/launch_flux_asset_service.sh` so CUDA library paths are configured.
- Pixel assets should be served from the materialized package path for fast map and atlas bootstrap.

7 Risk Register

Risk	Mitigation
Long generation blocks the server	Keep generation and art in detached systemd user workers with pollable status files.
Stale global assets leak into a new package	Repack from an isolated revision workspace and filter assets by world ID and revision.
World appears before validation completes	Preserve the `validation_probe` gate and clear it only after successful headless launch.
Duplicate generated revisions crowd the catalog	Deduplicate server-side by seed, then world ID, then world name.
Map exists but is not rendered	Inject generated map paths into both config and scenario map grid before repack.
Pixel boot stalls on asset hydration	Prefer materialized static package assets over slow dynamic file serving.
FLUX silently fails and placeholders slip through	Start FLUX with the CUDA-aware wrapper and hardfail critical asset errors.

8 Conclusion

Agora 2.0's feature scope is best understood as a complete world compiler and runtime, not merely a prompt-to-image or prompt-to-JSON tool. The system earns the slogan "one sentence, one world" by preserving a chain of evidence from brief, to builder spec, to compiled JSON, to package DB, to asset manifests, to browser validation, to live DB-backed play.